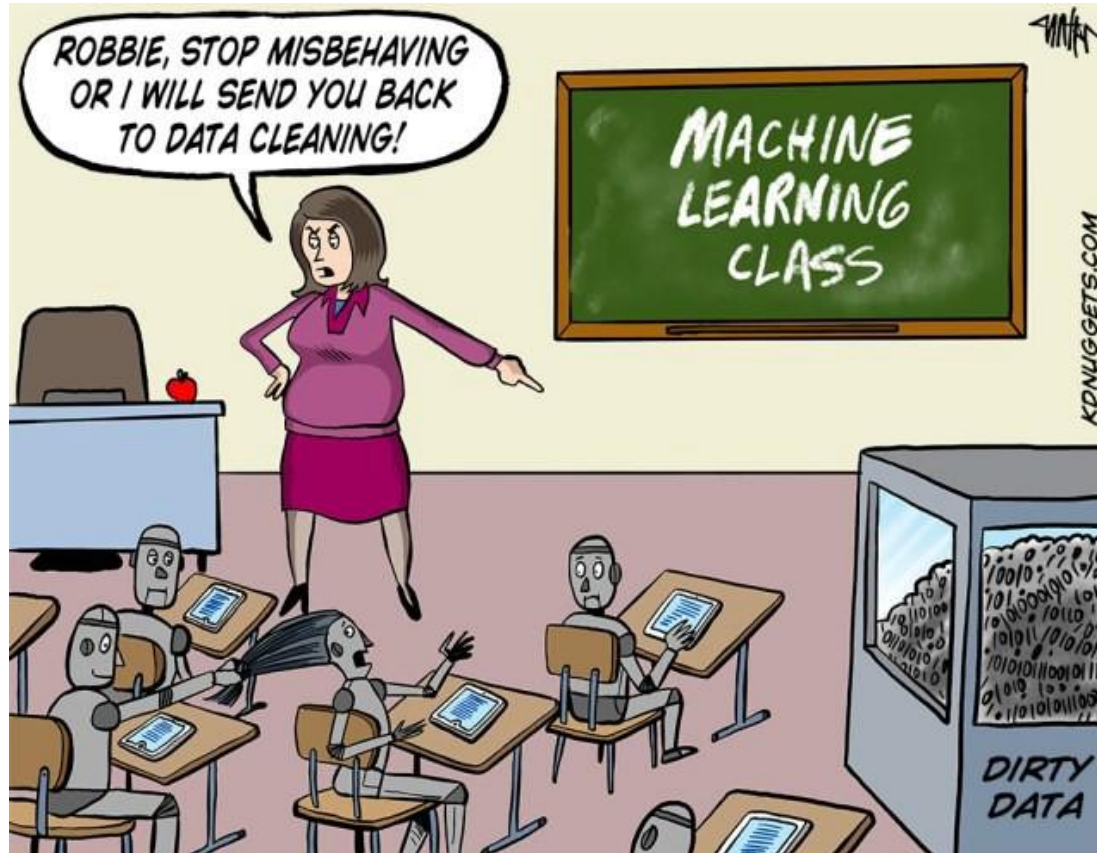


Dimensionality reduction

Machine learning

Doc. Ing. Radim Kolar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Introduction



Motivation - why dimensionality reduction

Today's applications generate a lot of data... and **features**. We can find application with only few features...and also with several **thousands** of features.

It's usually not feasible to analyze each and every variable at a microscopic level.

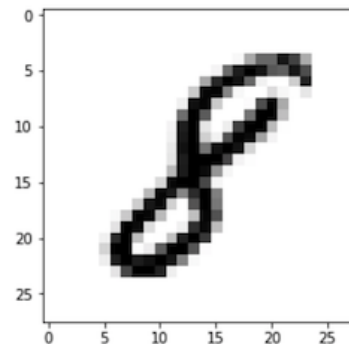
We need some mathematical tools to assess feature relevance, correspondence, validity...

There are many reasons to reduce the number of features to sufficient minimum, e.g. **computation complexity, generalization ability of the classifier** (for limited number of training patterns N , keeping the number of features as small as possible is in the line with our aim to design classifier with good generalization ability; see Curse of dimensionality)

Example

Consider images of digits with size 28x28 pixels.

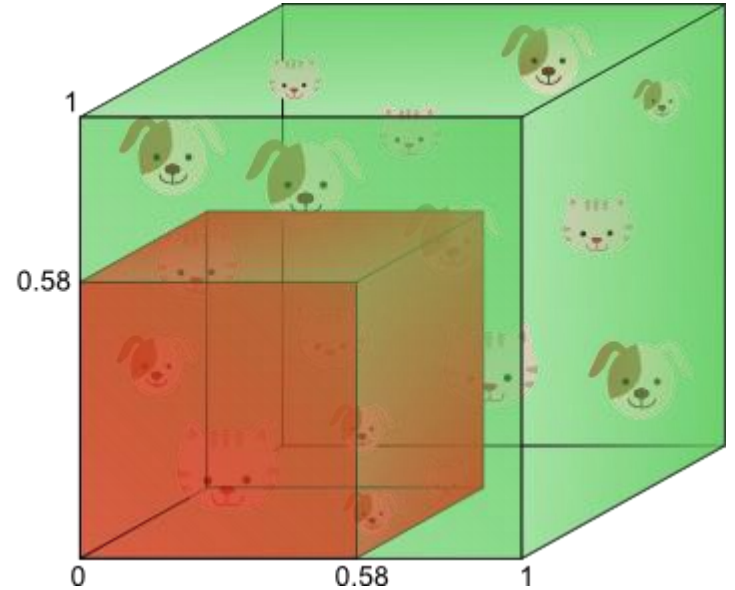
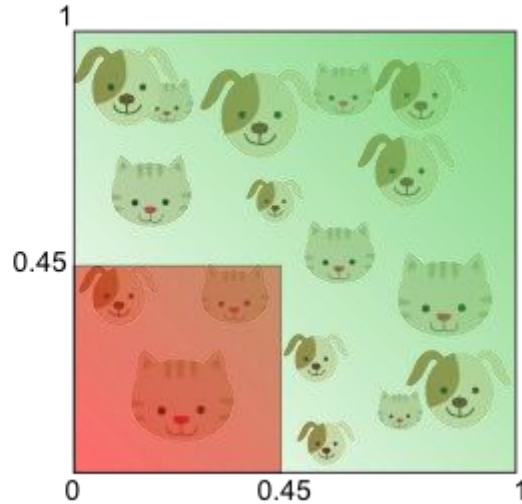
Representation of this digit image as a vector will result in $28 \times 28 = 784$ size of vector.



Dimensionality reduction

- Don't use *automatically* the tools for data analysis. It preferable to think and get know more about the features before analysis and explore data first, before building the model!
- Check feature redundancy.
- Improves the classification performance.
- Less dimensions lead to less computation/training time.
- Helps in visualizing data.
- Helps in data compression and reduced storage space.

Curse of Dimensionality



Curse of Dimensionality

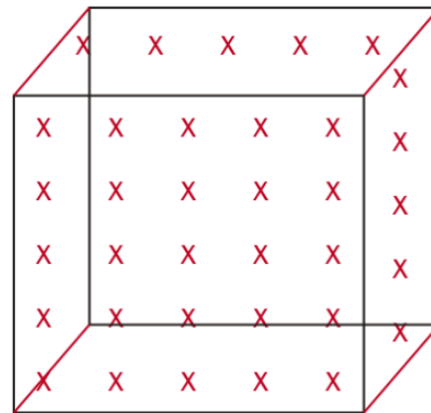
As the number of features grows, we need more data to reasonably span the N-dimensional space (i.e. to keep the same distance between samples). The number of samples grows exponentially - the example below: 5^1 , 5^2 , 5^3 ...



A one-dimensional features space
with five data points



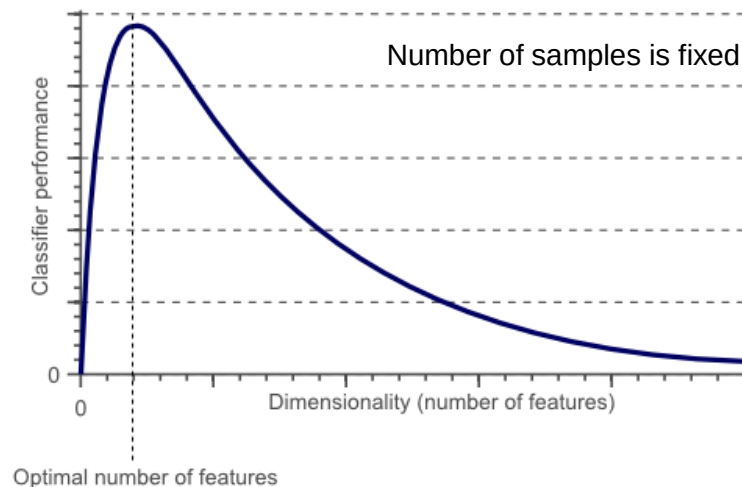
A two-dimensional features
space with 25 data points



A three-dimensional features space
with 125 data points

Curse of dimensionality - Hughes Phenomenon

Hughes phenomenon states that when you have a dataset of fixed number of samples, let's say 1000, then the more dimensions the samples have, the worse the system will be. For example if we have 2 dimensional feature vector, we will be able to train a decent predicting system with around 1000 samples, but when we have 500 dimensional samples, then 1000 samples won't be enough to train a good predicting system. This is also known as **curse of dimensionality**.



https://miro.medium.com/max/1020/0*iC_4D-LTTRUiZJeM.png

Curse of dimensionality - Hughes Phenomenon

The intuitive explanation is that to train a good predicting system it has to "see" a lot of possible combinations of possible values in order **to learn** the patterns in the data. With more dimensions there is more possible combinations and so you need many more samples.

Dimensionality reduction methods

These methods can be classified from different points of views. For example as **linear** approaches (PCA, LDA) or **nonlinear** (Kernel PCA, t-SNE).

If we think about features we can think about **features selection** (filter-based and wrapper-based approaches) and about **features reduction**.

Or they can be viewed as a **convex** (i.e. technique, which optimize an objective function that does not contain any local optima) and **non-convex** (objective functions do not contain a local optima).

Introduction

Now, we will look on different approach for analyzing the features and methods for features reduction and selection:

1. Basic analysis of the features
2. Selected measures of dependency
3. Feature reduction methods - PCA, kernel PCA, LDA, autoencoders
4. Feature selection methods - random tree, backward/forward selection, MRMR

First of all, let's look on the first step you should do if you have to analyze/classify a large number of data.

1. Basic analysis of the features

Basic analysis of the features

1. **Missing data** - check if there are some missing values for each variable. Discard some variable if missing data is above some threshold (e.g. 30%).
2. **Outlier removal** - an outlier is defined as a point that lies very far from the mean of corresponding variable. It might be a source of measurement error, but not always. In that case, proper cost function should be used.
3. **Low variance** - consider an extreme case - a variable in our dataset where all the observations have the same value, say 1. If we use this variable, do you think it can improve the model we will build? No, because this variable will have zero variance. Therefore - filter out the variables with low variances.
4. **High dependency** - high dependency between two variables means they have similar trends and are likely to carry similar information. We can calculate e.g. the correlation coefficient between independent numerical variables and if it crosses a certain threshold value, we can drop one of the variables. As a general guideline, we should keep those variables which show higher correlation with the target variable.

2. (Selected) Measures of dependency

(Selected) Measures of dependency

- A. Pearson correlation coefficient
- B. Spearman correlation coefficient
- C. Distance correlation
- D. Mutual information

A. Pearson correlation coefficient

It computes the relation between normalized values of two variables X and Y , with corresponding means μ and standard deviations σ :

$$\rho_{X,Y} = \frac{\mathbb{E}[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}$$

We need to subtract the mean and divide it by standard deviation due to different units in which the features are measured (seconds, meters, millimeters, mW...).

```
x = np.random.uniform(-1, 1, 100000)
print pearsonr(x, x**2)[0]
```


Remark - Data normalization

In many practical applications the features are measured within different units and ranges. Thus the features with large values may have a large influence in the cost function than features with low values, although it doesn't necessary reflect their respective significance in the design of the classifier.

A straightforward (not only one) technique is usually used. For a feature vector $\mathbf{X}$ the normalization is:

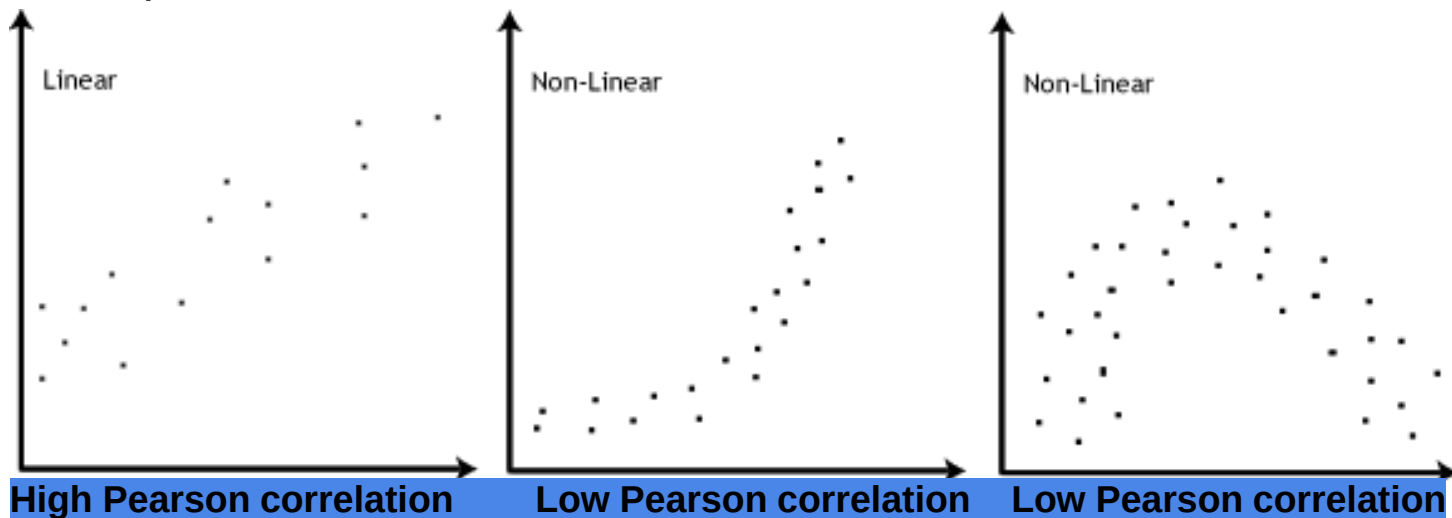
$$x_i^{norm} = \frac{x_i - \mu}{\sigma}$$

Where μ is a mean computed from feature samples and σ is corresponding standard deviation.

Other possible normalization: range 0 - 1, range +1:-1, extraction of mean only...

A. Pearson correlation

One obvious drawback of Pearson correlation as a feature ranking mechanism is that it is only **sensitive to a linear relationship**; i.e. Pearson's correlation determines the degree to which a relationship is linear. If the relation is non-linear, Pearson correlation can be close to **zero** even if there is a “*one-to-one*” (i.e. very close) correspondence between the two variables.



B. Spearman correlation coefficient

The Spearman's Rank Correlation Coefficient is the non-parametric statistical measure used to study the strength of association between the two ranked variables. This method is applied to the ordinal set of numbers, which can be arranged in order, i.e. one after the other so that ranks can be given to each. In the rank correlation coefficient method, the ranks are given to each individual on the basis of its quality or quantity, such as ranking starts from position 1st and goes till Nth position for the one ranked last in the group. The formula to calculate the rank correlation coefficient R is:

$$R = \frac{(1 - 6 \sum D^2)}{N (N^2 - 1)} = \frac{(1 - 6 \sum D^2)}{N^3 - N}$$

D - difference of ranks, N - number of observations.

Example: <https://accendoreliability.com/spearman-rank-correlation-coefficient/>

C. Distance correlation measure

Distance correlation is a measure of dependence between two random vectors **X** and **Y** (samples of features), and its value ranges from zero to one. It uses Euclidean distance in its formula, and it has the main properties:

It is zero **if and only if** the two random vectors are **independent**, which means that obtaining zero value characterizes the independence between the two vectors.

For some linear or non-linear dependence, the value is greater than 0.

Distance correlation measure

Sample distance correlation coefficient can be practically computed simply. Let x_k and y_k ($k = 1, 2, \dots, N$) be samples from some measurement (e.g. we measure temperature and pressure at the same time). Therefore we get a real valued vector of random variables (X, Y) . First, compute the N by N **distance matrices** $(a_{j,k})$ and $(b_{j,k})$ containing all pairwise distances:

$$a_{j,k} = |x_j - x_k|$$
$$b_{j,k} = |y_j - y_k|$$

i.e. for each sample we compute the distance to all samples.

Now, we can compute the correlation between these matrices, but firstly we should extract the mean values.

Distance correlation measure

From each entry of distance matrices we subtract the $\bar{a}_{j.}$ (the j -th row mean), $\bar{a}_{.k}$ (the k -th column mean), and $\bar{a}_{..}$ is the *grand* (total) mean of the distance matrix of the X sample. Similar for distance matrix b . We must add the grand mean because we subtract twice the *vector* (row, column) means:

$$A_{j,k} = a_{j,k} - \bar{a}_{j.} - \bar{a}_{.k} + \bar{a}_{..}$$

$$B_{j,k} = b_{j,k} - \bar{b}_{j.} - \bar{b}_{.k} + \bar{b}_{..}$$

Distance correlation measure

The **squared sample covariance** is then defined as:

$$\text{dCov}_n^2(X, Y) := \frac{1}{N^2} \sum_{j=1}^N \sum_{k=1}^N A_{j,k} B_{j,k}.$$

And finally the distance correlation measure:

$$\text{dCor}(X, Y) = \frac{\text{dCov}(X, Y)}{\sqrt{\text{dVar}(X) \text{dVar}(Y)}},$$

Where the variances (or standard deviations) are used to “normalize” the data (they are measured in different units).

Matlab Code

Generate data

```
N = 1000;  
x = randn(1,N);  
y = x.*x;  
  
x = x(:);  
y = y(:);  
  
figure(1);  
plot( x, y, 'r*' );
```

Compute A_{jk} , B_{jk}

```
a = zeros( N, N );  
b = zeros( N, N );  
for ii = 1:N  
    a(ii,:) = abs(x(ii) - x);  
end  
for ii = 1:N  
    b(ii,:) = abs(y(ii) - y);  
end  
  
RowMean = mean(a,1);  
ColumnMean = mean(a,2);  
a = bsxfun(@minus, a, RowMean );  
a = bsxfun(@minus, a, ColumnMean );  
a = a + mean( RowMean );  
  
RowMean = mean(b,1);  
ColumnMean = mean(b,2);  
b = bsxfun(@minus, b, RowMean );  
b = bsxfun(@minus, b, ColumnMean );  
b = b + mean( RowMean );
```

Compute the coefficient

```
% Distance correlation  
dcov = sum(sum(a.*b))/(N^2)  
dvarx = sum(sum(a.*a))/(N^2)  
dvary = sum(sum(b.*b))/(N^2)  
  
dcor = sqrt(dcov/sqrt(dvarx*dvary))
```


D. Mutual information

Another, more robust option for correlation estimation is mutual information. This measure is used in many areas of machine learning, data processing, image registration etc.

For in-depth information about MI and feature selection see: [Vergara et al. DOI: 10.1007/s00521-013-1368-0](https://doi.org/10.1007/s00521-013-1368-0)

D. Mutual information

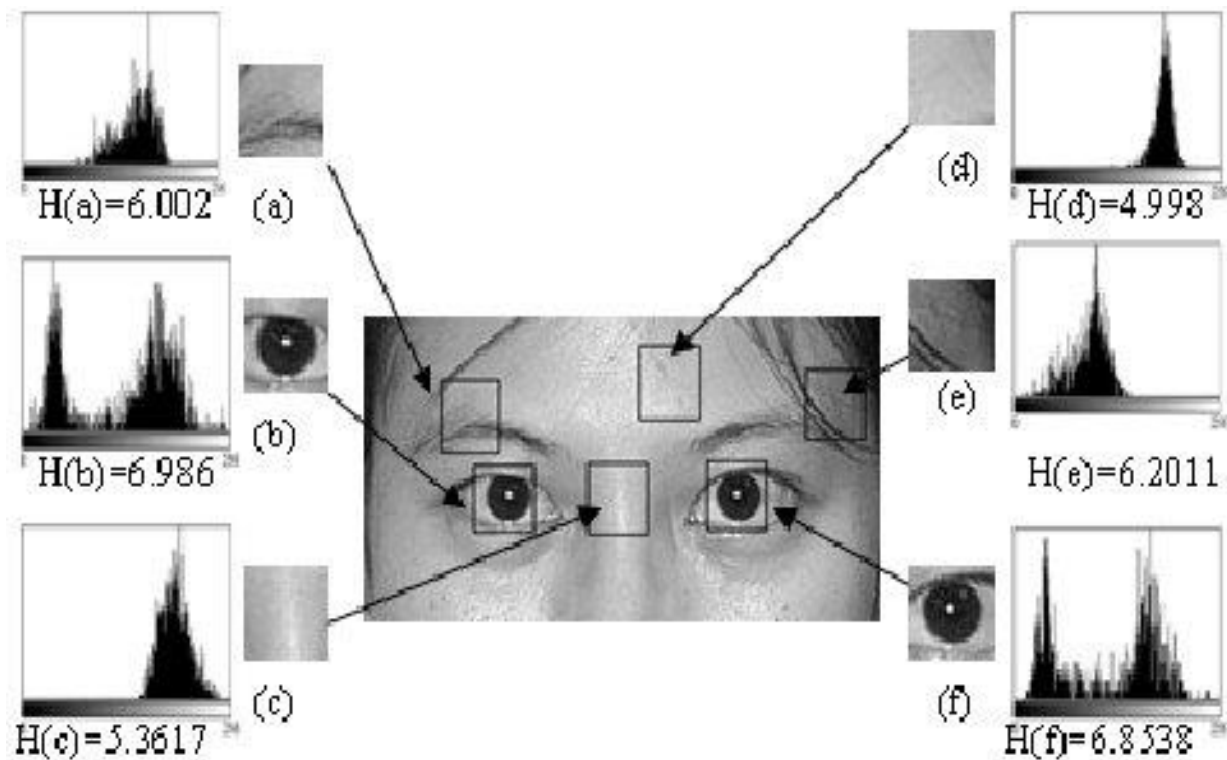
We will start with entropy. Formally, the entropy of a discrete random variable x , with probability mass function $p(x(i)) = \Pr\{x=x(i)\}$ is defined as:

$$H(x) = - \sum_{i=1}^n p(x(i)) \log_2(p(x(i)))$$

Entropy (H) is a measure of uncertainty of a random variable. The uncertainty is related to the probability of occurrence of an event. Intuitively, **high entropy means that each event has about the same probability of occurrence, while low entropy means that each event has a different probability of occurrence.**

Note: A **probability mass function** differs from a **probability density function** (PDF) in that the PDF is associated with continuous rather than discrete random variables. Formally, the values of the PDF are not probabilities as such: a PDF must be integrated over an arbitrary interval to yield a probability.

Entropy



Mutual information

Let $\mathbf{x}$ and $\mathbf{y}$ be two random discrete variables. The **joint entropy** of $\mathbf{x}$ and $\mathbf{y}$, with **joint mass probability** $p(\mathbf{x}(i), \mathbf{y}(j))$, is the sum of the uncertainty contained by the two variables. Formally, joint entropy is defined as follows:

$$H(\{x, y\}) = - \sum_{i=1}^n \sum_{j=1}^n p(x(i), y(j)) \cdot \log_2(p(x(i), y(j)))$$

The joint entropy has values in the range,

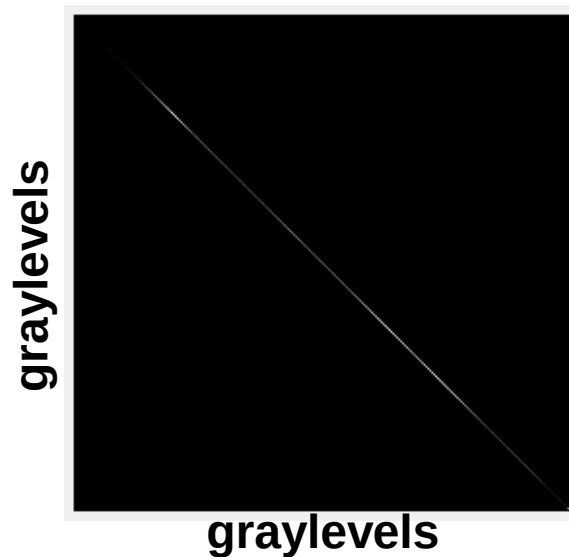
$$\max(H(x), H(y)) \leq H(\{x, y\}) \leq H(x) + H(y)$$

The maximum value of this inequality happens when x and y are completely independent. The minimum value occurs when x is completely dependent on y .

Mutual information

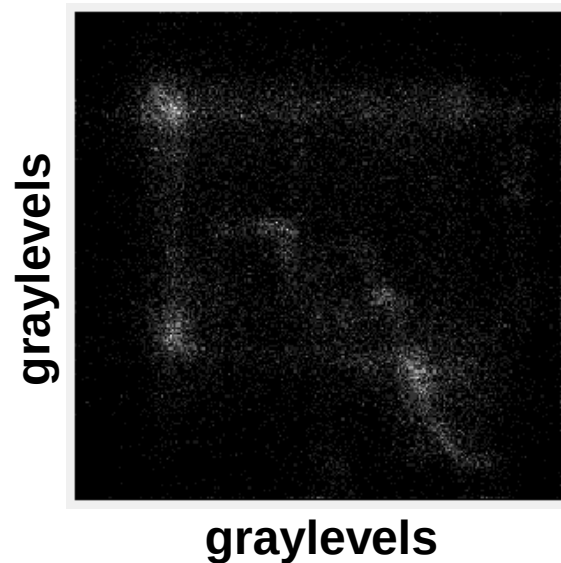
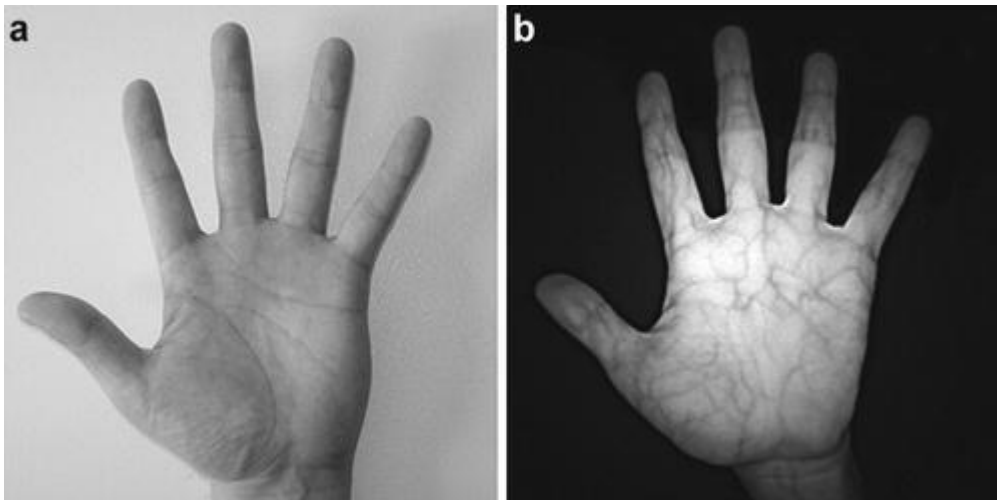
How a joint histogram of two completely identical processes will look like?

Consider an image as source of samples of one random process and the copy of this image as a second random process.



Mutual information

Consider this situation: x - the same; y – different modality but ‘somehow’ dependent on (similar to) x .



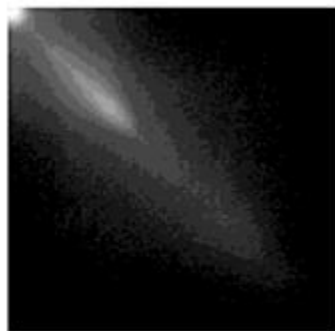
Mutual information

Influence of joint histogram on joint entropy.

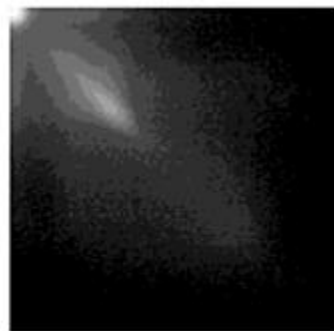
Examples:



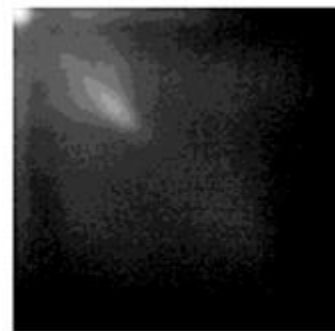
3.82



6.79



6.98



7.15

Joint entropy values below the images.

Mutual information

The mutual information $I(x,y)$ is a measure of the amount of information that one random variable has about another variable. It can be expressed with entropies:

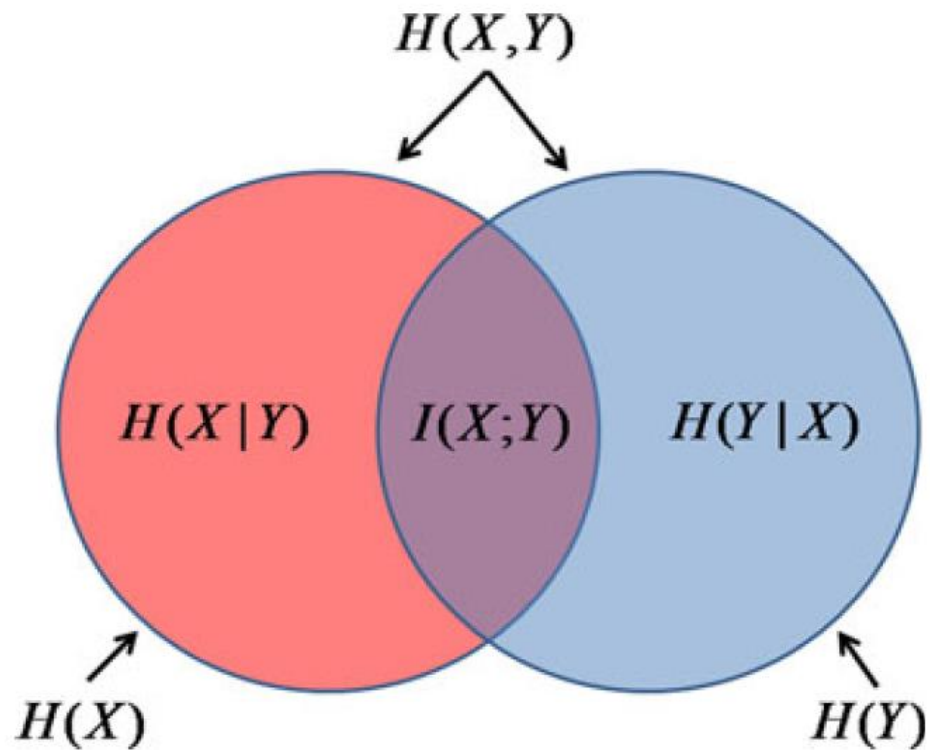
$$I(x;y) = H(x) + H(y) - H(x,y)$$

Definition using probabilities:

$$I(x; y) = \sum_{i=1}^n \sum_{j=1}^n p(x(i), y(j)) \cdot \log \left(\frac{p(x(i), y(j))}{p(x(i)) \cdot p(y(j))} \right)$$

where $I(x,y)$ is zero when x and y are statistically independent: $p(x(i),y(i)) = p(x).p(y)$.

Mutual information and entropies



Mutual information - problems

It can be inconvenient to use directly for feature ranking for two reasons though.

Firstly, the MI values can be incomparable between two datasets.

Secondly, it can be inconvenient to compute for continuous variables: in general the variables need to be discretized by binning, but the mutual information score can be quite sensitive to bin selection.

Maximal information coefficient is a technique developed to address these shortcomings. It searches for optimal binning and turns mutual information score into a metric that lies in range $[0;1]$.

Maximal information coefficient - MIC

[Reshef et al.](#) defined MIC as mutual information estimate $I(x;y)$ computed using a data-dependent binning scheme. Let n_x and n_y , respectively, denote the number of bins imposed on the x and y axes. The MIC binning scheme is chosen so that:

(1) the total number of bins ($n_x.n_y$) does not exceed some user-specified value B ;

(2) the value of the ratio

$$MIC(x, y) = \frac{I(x,y)}{Z_{MIC}}$$

is maximized; where $Z_{MIC} = \log_2(\min(n_x, n_y))$.